Guía de Estudio JavaScript (Temas 3-7)

Tema 3: Estructuras de Datos (Arrays, Sets, Maps)

En este tema se estudian las formas de organizar y almacenar colecciones de datos. Los **Arrays** son listas ordenadas, los **Sets** son colecciones de valores únicos y los **Maps** son diccionarios que almacenan pares de clave-valor.

Es una lista numerada (0, 1, 2...) donde el orden importa.

Código: let frutas = ["Manzana", "Pera", "Uva"];

Arrays (Lista Ordenada)

- **let arr = [] / new Array()**: Crea un array vacío.
- **arr.length**: Devuelve el número total de elementos del array.
- **arr[i]**: Accede al elemento en la posición i (empieza en 0).
- **arr.push(elemento)**: Añade un elemento al final y devuelve la nueva longitud.
- **arr.pop()**: Elimina y devuelve el último elemento del array.
- **arr.unshift(elemento)**: Añade un elemento al principio del array.
- **arr.shift()**: Elimina y devuelve el primer elemento del array.
- **arr.splice(pos, n)**: Elimina n elementos desde la posición pos (también puede insertar).
- **arr.slice(inicio, fin)**: Devuelve una copia de una parte del array sin modificar el original.
- **arr.concat(otroArray)**: Une dos arrays en uno nuevo.
- **arr.indexOf(elemento)**: Devuelve el índice de la primera aparición del elemento (-1 si no existe).
- **arr.lastIndexOf(elemento)**: Busca el índice del elemento comenzando desde el final.
- **arr.includes(elemento)**: Devuelve true si el elemento existe en el array.
- **arr.sort()**: Ordena los elementos (alfabéticamente por defecto).
- **arr.reverse()**: Invierte el orden de los elementos.
- **arr.join(separador)**: Une los elementos en un texto separado por el carácter indicado.
- **arr.forEach(funcion)**: Ejecuta una función por cada elemento del array.
- **arr.map(funcion)**: Crea un nuevo array transformando cada elemento con la función.
- **arr.filter(funcion)**: Crea un nuevo array solo con los elementos que cumplen la condición.
- **arr.reduce(funcion)**: Reduce el array a un único valor acumulado.

Set (Conjunto Único)

Es una colección de valores donde no se permiten duplicados; si añades un valor repetido, lo ignora automáticamente.

Es una bolsa de valores donde no se permite repetir nada. Si metes un duplicado, lo ignora.

Código: let unicos = new Set([1, 5, 5, 5, 8]); // El resultado real es {1, 5, 8}

- **new Set()**: Crea un conjunto nuevo (no admite duplicados).
- **set.add(valor)**: Añade un valor al conjunto.
- **set.delete(valor)**: Elimina un valor específico.
- **set.has(valor)**: Devuelve true si el valor existe en el conjunto.
- **set.clear()**: Elimina todos los elementos.
- **set.size**: Propiedad que indica cuántos elementos tiene el conjunto.

Maps (Mapas)

Es una colección de pares clave-valor que permite usar cualquier tipo de dato (incluso objetos) como clave y recuerda el orden de inserción

Es como una agenda telefónica: buscas un nombre (clave) para obtener su número (valor).

Código: let precios = new Map(); precios.set("Pan", 0.80);

- **new Map()**: Crea un nuevo mapa para guardar pares clave-valor.
- **map.set(clave, valor)**: Guarda un valor asociado a una clave.
- **map.get(clave)**: Recupera el valor asociado a esa clave.
- **map.delete(clave)**: Borra el elemento de esa clave.
- **map.has(clave)**: Devuelve true si la clave existe.
- **map.keys()**: Devuelve un iterador con las claves.
- **map.values()**: Devuelve un iterador con los valores.

Tema 4: Funciones

Las funciones son bloques de código reutilizables que permiten la modularidad. Se pueden definir de varias formas (declarativa, expresión, flecha) y manejan datos a través de parámetros y valores de retorno.

- **function nombre() {}**: Define una función de forma declarativa (sufre hoisting).
- **const nombre = function() {}**: Define una función por expresión (anónima en variable).
- **() => {}**: Función flecha (Arrow Function), sintaxis moderna y concisa.
- **return valor**: Devuelve un resultado y detiene la función.
- **function(p=valor)**: Define un parámetro con valor por defecto.
- **function(...args)**: Parámetro Rest, agrupa argumentos infinitos en un array.
- **arguments**: Objeto disponible en funciones clásicas con todos los parámetros recibidos.
- **(function(){})()**: Función autoejecutable (IIFE), se lanza al definirse.

- **callback**: Función pasada como argumento para ser ejecutada por otra función.

Tema 5: POO y Objetos Predefinidos

Este tema cubre la Programación Orientada a Objetos, basada en clases y prototipos para modelar la realidad. También incluye el formato JSON para intercambio de datos y objetos nativos como Date o Math.

Clases y Objetos

- **class Nombre {}**: Define una plantilla (clase) para crear objetos.
- **constructor()**: Método que se ejecuta automáticamente al crear una instancia.
- **new NombreClase()**: Crea un nuevo objeto (instancia) basado en una clase.
- **this**: Hace referencia al objeto actual dentro de la clase.
- **extends**: Indica que una clase hereda propiedades de otra.
- **super()**: Llama al constructor de la clase padre.
- **static**: Define un método que se llama desde la clase, no desde el objeto.
- **instanceof**: Comprueba si un objeto pertenece a una clase.

JSON

Es un formato de texto ligero basado en la sintaxis de objetos JS, utilizado para intercambiar datos entre el cliente y el servidor.

Es un formato de texto (string) que parece código JS, usado para enviar datos por internet.

Código: let datos = '{"usuario": "Ana", "id": 123}';

- **JSON.stringify(obj)**: Convierte un objeto JavaScript a texto JSON.
- **JSON.parse(texto)**: Convierte texto JSON a un objeto JavaScript.

Objetos Predefinidos

- **new Date()**: Crea un objeto con la fecha y hora actual.
- **getDate() / getMonth()**: Obtiene el día del mes y el mes (0-11).
- **getFullYear()**: Obtiene el año completo.
- **Math.random()**: Genera un número aleatorio entre 0 y 1.
- **Math.floor(x)**: Redondea un número hacia abajo.
- **Math.ceil(x)**: Redondea un número hacia arriba.
- **Math.round(x)**: Redondea al entero más cercano.
- **Math.max(a,b) / Math.min(a,b)**: Devuelve el mayor o menor de los números.

Tema 6: Modelo de Objetos (BOM y DOM)

Se diferencia entre el BOM (navegador, ventanas, historial) y el DOM (estructura HTML de la página). Permite seleccionar, modificar y crear elementos visuales dinámicamente.

BOM (Navegador)

Es el conjunto de objetos que permite a JavaScript interactuar con la ventana del navegador (historial, URL, pantalla) y no con el contenido HTML

Controla la ventana, el historial o la barra de direcciones (cosas fuera de la página web en sí).

Código: window.alert("¡Hola!"); o window.history.back();

- **window**: Objeto global que representa la ventana del navegador.
- **window.innerWidth**: Ancho de la ventana (viewport).
- **window.alert(msg)**: Muestra una alerta visual.
- **window.confirm(msg)**: Pide confirmación (Aceptar/Cancelar).
- **window.prompt(msg)**: Pide al usuario que introduzca un texto.
- **setTimeout(func, ms)**: Ejecuta una función una vez tras un tiempo.
- **setInterval(func, ms)**: Ejecuta una función repetidamente cada cierto tiempo.
- **navigator.userAgent**: Información sobre el navegador y sistema operativo.
- **location.href**: Contiene la URL actual (se puede modificar para redirigir).
- **history.go(-1)**: Vuelve a la página anterior en el historial.

DOM (Documento)

Es la representación del documento HTML como un árbol de nodos, permitiendo modificar el contenido, estructura y estilo de la página

Es el árbol de etiquetas HTML de tu página que puedes modificar.

Código: document.body.style.background = "red";

- **document.getElementById(id)**: Selecciona un único elemento por su ID.
- **document.querySelector(css)**: Selecciona el primer elemento que coincida con el selector.
- **document.querySelectorAll(css)**: Selecciona todos los elementos que coincidan (NodeList).
- **elem.innerHTML**: Lee o modifica el contenido HTML interno.

- **elem.textContent**: Lee o modifica solo el texto (sin etiquetas).
- **elem.style.propiedad**: Cambia estilos CSS en línea (ej: .color).
- **elem.classList.add('clase')**: Añade una clase CSS al elemento.
- **elem.classList.toggle('clase')**: Alterna (pone/quita) una clase CSS.
- **elem.getAttribute(nombre)**: Obtiene el valor de un atributo HTML.
- **elem.setAttribute(nombre, valor)**: Cambia el valor de un atributo HTML.
- **document.createElement(tag)**: Crea un nuevo elemento HTML en memoria.
- **padre.appendChild(hijo)**: Inserta un elemento hijo al final del padre.
- **padre.removeChild(hijo)**: Elimina un elemento hijo del DOM.

Tema 7: Eventos

Los eventos permiten que la página reaccione a las acciones del usuario (clics, teclado, formularios). Se gestionan mediante "escuchadores" que ejecutan código cuando ocurre el suceso.

Gestión de Eventos

- **elem.addEventListener('evento', func)**: Asocia una función a un evento.
- **elem.removeEventListener('evento', func)**: Elimina la asociación de un evento.

Objeto Evento (e)

- **e.target**: Elemento específico que originó el evento.
- **e.preventDefault()**: Cancela el comportamiento por defecto (ej: envío de form).
- **e.stopPropagation()**: Detiene la propagación del evento a los padres.
- **e.key**: Nombre de la tecla pulsada en eventos de teclado.
- **e.clientX / e.clientY**: Coordenadas del ratón en la ventana.

Tipos de Eventos

- **click**: Se activa al pulsar y soltar el ratón sobre un elemento.
- **mouseover**: El ratón entra en la zona del elemento.
- **mouseout**: El ratón sale de la zona del elemento.
- **keydown**: Se pulsa una tecla (antes de soltarla).
- **submit**: Se intenta enviar un formulario.
- **change**: El valor de un input cambia y pierde el foco.
- **focus**: El elemento se activa o selecciona.
- **blur**: El elemento pierde la activación o selección.
- **DOMContentLoaded**: El HTML inicial se ha cargado completamente.
- **load**: Toda la página (imágenes incluidas) ha terminado de cargar.

Escuchar (Listener): Se usa para detectar cuando ocurre la acción.

Código: boton.addEventListener('click', () => alert("¡Hola!"));

Objeto Evento (e): Contiene información detallada de lo que pasó (qué tecla fue, coordenadas del ratón, etc.).

Código: document.addEventListener('keydown', (e) => console.log("Tecla pulsada: " + e.key));

Prevenir Comportamiento (preventDefault): Evita que el navegador haga su acción por defecto (como enviar un formulario o seguir un enlace).

Código: enlace.addEventListener('click', (e) => e.preventDefault()); // No cambia de URL

Detener Propagación (stopPropagation): Evita que el evento "suba" (burbujeo) a los elementos padre.

Código: boton.addEventListener('click', (e) => e.stopPropagation()); // El div padre no se entera

---

Formularios (Forms)

Los formularios son la principal vía de entrada de datos en las aplicaciones web. JavaScript permite acceder a los formularios y sus controles, leer y modificar sus valores, y controlar el envío (validación) mediante eventos.

Acceso y Control

- **document.forms:** Colección que contiene todos los formularios del documento.
- **document.forms[i] / document.forms['nombre']:** Accede a un formulario específico por su índice o por su atributo name.
- **form.elements:** Colección de todos los controles (inputs, botones, selects) dentro del formulario.
- **form.length: Devuelve el número de controles que tiene el formulario.**
- **form.submit():** Envía los datos del formulario manualmente (como si se pulsara enviar).
- **form.reset():** Devuelve todos los campos del formulario a sus valores iniciales.

Propiedades de los Inputs (Controles)

- **input.value:** Obtiene o define el valor actual del control (texto, número, etc.).
- **input.name:** Obtiene el nombre del control.
- **input.type:** Obtiene el tipo de control (text, checkbox, password...).
- **input.checked:** Devuelve true si un checkbox o radio button está marcado, o false si no.
- **input.disabled:** Si es true, el control está desactivado y no se puede usar.
- **input.focus():** Pone el foco (cursor activo) en el control.
- **input.blur():** Quita el foco del control.

Eventos de Formulario

- **submit:** Se dispara al intentar enviar el formulario. Es fundamental usar **preventDefault()** aquí para validar antes de enviar.
- **reset:** Se dispara cuando se reinicia el formulario.
- **focus:** Se activa cuando un control recibe el foco (el usuario entra en él).
- **blur:** Se activa cuando un control pierde el foco (el usuario sale de él).
- **change:** Se activa cuando el valor cambia y el usuario retira el foco del control.